A Transformer Optimized Planner for Autonomous Vehicle On-Ramping Merging Task

Ping Lu, Hui Xu, and Bo Hu

***Abstract*—Motion planning is the core component of autonomous driving technology. Traditional rule- and optimization-based methods face limitations when handling long-tail scenarios and complex traffic dynamics. While learning-based approaches offer potential solutions, they often compromise on trajectory safety and interpretability. In this context, this article introduces a transformer optimized planner (TOP), which combines generative artificial intelligence (AI) with optimization methods for autonomous driving on-ramping merging task. Specifically, TOP treats motion planning tasks as an offline reinforcement learning (RL) problem and employs the powerful sequence modeling capabilities of transformers, typically used in natural language processing (NLP), to derive an initial planning trajectory that maximizes utility from a fixed, previously collected dataset. Additionally, TOP facilitates real-time planning by utilizing efficient tokenization and temporal abstraction to compress input sequences into high-level latent actions. On this basis, optimization methods are employed to refine and safely adjust the transformer-generated trajectory, ensuring it adheres to safety requirements and vehicle kinematic constraints. To evaluate the effectiveness of the proposed algorithm, we performed simulations and conducted experiments using the Apollo industrial control unit, comparing the performance of TOP with the transformer- and optimization-based planner in an on-ramp merging scenario. It is found that TOP guarantees that the motion planning decisions are made within tight latency constraints (such as 100 ms target) even in complex scenarios, demonstrating 33 times shorter online execution time compared to the transformer-based planner and was nearly three times faster than the optimization-based planner while ensuring safety and performance.**

***Index Terms*—Motion planning, offline reinforcement learning (RL), real time, transformer.**

I. Introduction

MOTION planning in autonomous vehicles refers to the process of determining a trajectory that the vehicle should follow to reach its destination safely, smoothly, and efficiently while avoiding obstacles and adhering to traffic rules. Among various complex scenarios, on-ramp merging poses a significant challenge for autonomous vehicles as it requires navigating through dense and unpredictable traffic, accurately predicting the actions of other drivers, identifying safe gaps, and making real-time decisions that balance safety, efficiency, and comfort [1].

Current motion planning methods can be broadly categorized into three types: rule-, optimization-, and learning-based methods. Rule-based methods rely on predefined rules to determine the vehicle's future trajectory. For example, Bouchard et al. [2] developed a practical rule-based motion planner for autonomous vehicles, using a two-layer rule-based theory. The first layer determines a set of feasible parametrized behaviors, given the perceived state of the environment. From this, a resolution function chooses the most conservative high-level maneuver. The second layer then reconciles the parameters into a single behavior, which is then used by the trajectory planner to produce a safe and smooth trajectory. Although this type of method offers high interpretability, the rule set tends to expand significantly with the increasing complexity and diversity of driving scenarios, making maintenance challenging. Optimization methods seek the optimal solution by minimizing the objective function while satisfying various constraints. For instance, the EM Planner in Baidu's Apollo open-source platform initially generates a trajectory using dynamic programming (DP) optimizer and then refines it through quadratic programming (QP) optimizer, with each method addressing different constraints [3]. Wang et al. [4] proposed a hybrid automaton architecture for the merging and splitting of connected and automated vehicle (CAV) platoons, utilizing model predictive control (MPC) and velocity obstacle (VO) theory to achieve collision avoidance. Bhattacharyya et al. [5] proposed an adaptive interactive mixed-integer MPC method for highway merging scenarios. The method utilizes mixed-integer quadratic programming (MIQP) to address nonconvex constraints, such as lane selection and collision avoidance, and integrates inverse optimal control to estimate the cost weights of neighboring vehicles in real time. Experimental results show that this approach significantly improves safety and efficiency in complex merging scenarios. Although the aforementioned

Received 18 September 2024; revised 15 February 2025 and 20 April 2025; accepted 28 May 2025. Date of publication 14 July 2025; date of current version 21 November 2025. This work was supported in part by the Natural Science Foundation of Chongqing under Grant CSTB2023NSCQ-MSX0766 and Grant CSTB2023NSCQ-MSX0418. *(Corresponding author: Bo Hu.)*

The authors are with the Key Laboratory of Advanced Manufacturing Technology for Automobile Parts, Ministry of Education, Chongqing University of Technology, Chongqing 400054, China (e-mail: luping@stu.cqut.edu.cn; XuH@stu.cqut.edu.cn; b.hu@cqut.edu.cn).

Digital Object Identifier 10.1109/TIE.2025.3579052

1557-9948 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: MULTIMEDIA UNIVERSITY. Downloaded on December 08,2025 at 09:01:57 UTC from IEEE Xplore. Restrictions apply.

optimization-based methods provide strong safety guarantees by ensuring trajectories adhere to predefined constraints, their reliance on manual tuning of cost function weights and high computational complexity increases with the complexity and diversity of driving scenarios, making them difficult to scale and adapt in real-world applications.

Learning-based methods can leverage vast amounts of driving data collected and stored in the cloud. This approach offers a potential way to continuously learn from collected data while capturing interaction patterns between the ego vehicle and other traffic participants [6]. As a result, the learned trajectories are better suited to complex interactive environments and more closely resemble human driving behavior. For example, Chen et al. [7] proposed a motion planner based on deep imitation learning (IL), which aims to minimize the difference between the human expert driving trajectories and the trajectories predicted by the IL motion planner. However, this approach requires gathering large quantities of high-quality expert driving data, which is both difficult and costly. Additionally, because the IL motion planner relies on imitation rather than exploration, its strategies often struggle to surpass those of the experts, making it challenging to handle highly complex or dynamically changing tasks that require knowledge beyond the expert data. Recently, the transformer model [8], as the core architecture for large language models (LLMs) such as the ChatGPT series, has achieved remarkable success in the field of natural language processing (NLP). Motivated by this success, the transformer model has also exhibited remarkable potential in autonomous driving field. For example, Mao et al. [9] fine-tuned ChatGPT-3.5 using an autonomous driving dataset. They converted the vehicle's surrounding perception data into a format understandable by the LLM, enabling it to predict the vehicle's trajectory for the upcoming seconds. However, due to the limitations of the ChatGPT-3.5 API, the inference time remains unknown, and the safety performance cannot be assured. Furthermore, while LLMs exhibit strong language comprehension and commonsense reasoning skills, their ability to perform accurate numerical reasoning has yet to be established [10]. In addition, instead of directly fine-tuning a mature LLM with domain-specific datasets, one can also use the transformer architecture that underlies LLMs to train a model from scratch. For example, Ngiam et al. [11] proposed scene transformer, a transformer-based model for jointly predicting and planning the motion trajectories of multiple agents in autonomous driving scenarios. It implicitly models interactions between agents and generates consistent future trajectory predictions. However, when the number of traffic participants in the scenario increases, the inference speed of scene transformer significantly decreases, resulting in insufficient real-time performance and difficulty meeting the real-time requirements of autonomous driving systems. Additionally, Huang et al. [12] introduced Game Former, a transformer-based interactive prediction and planning model that employs hierarchical game theory to model multiagent interactions, significantly improving prediction and planning performance. However, the hierarchical decoding structure of this model increases computational complexity. During multilevel reasoning, inference time rises significantly with the number of decoding layers, potentially limiting its application in real-time systems. Moreover, these two transformer-based methods do not derive optimal trajectories directly from datasets, favoring common but suboptimal actions and potentially missing higher reward, less frequent behaviors. This limits their ability to generate optimal trajectories, especially in novel scenarios. In contrast, transformer-based approaches combined with offline reinforcement learning (RL) offer a promising alternative. For example, Janner et al. [13] drew inspiration from the transformer architecture behind generative pretrained transformer (GPT) and effectively applied it to the field of offline RL. They treated trajectory prediction as a sequence modeling problem, leveraging the powerful capabilities of transformers to capture the intricate relationships between states, actions, and rewards in trajectories, thereby generating high-return trajectory sequences. However, to the authors' best knowledge, this approach has not yet been applied to motion planning tasks for autonomous driving and similar to the aforementioned transformer-based methods that rely on an autoregressive generation approach without a strict safety guarantee, its slower inference speed and uncertain safety performance in unfamiliar scenarios may limit its effectiveness in motion planning for autonomous vehicles.

Integrating learning-based methods with optimization-based approaches appears to be a promising avenue for the future of motion planning. Optimization methods, which rely on explicit, predefined rules, provide strong safety guarantees by ensuring performance not fall below a certain lower limit. However, they are computationally demanding and can have difficulty handling complex, dynamic scenarios. In contrast to optimization-based approaches, learning-based methods excel in handling complex and dynamic scenarios by learning directly from offline data. This enables them to achieve superior real-time performance in familiar training contexts online. However, using learning-based methods alone may lack sufficient safety guarantees for unknown environments. By combining learning-based methods with optimization techniques, high-quality initial trajectories can be quickly generated through learning-based approaches, providing a strong foundation for further refinement using optimization methods. This will enhance overall performance, reduce computational complexity, and improve real-time capabilities. In this context, we propose a trajectory optimized planner (TOP), which treats the motion planning problem as an offline RL task and optimizes the overall performance by combining the ego vehicle's motion planning with other vehicles' trajectory prediction. The contributions of this article are as follows.

1) The sequence modeling capabilities of transformers and offline RL techniques are employed in autonomous vehicle motion planning tasks to generate an initial planning trajectory that maximizes utility from a precollected dataset.
2) Efficient tokenization and temporal abstraction techniques are introduced to significantly enhance the computational efficiency of the proposed transformer-based motion planning method while maintaining trajectory performance.
3) Optimization methods are incorporated into the transformer-generated trajectory to ensure compliance with

Authorized licensed use limited to: MULTIMEDIA UNIVERSITY. Downloaded on December 08,2025 at 09:01:57 UTC from IEEE Xplore. Restrictions apply.

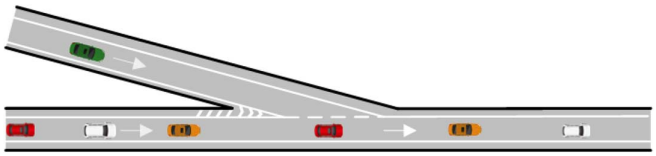

Fig. 1. Schematic of the driving scenario.

predefined safety requirements and vehicle kinematic constraints.

II. Driving Scenario and Problem Formulation

A. Driving Scenario

As shown in Fig. 1, the on-ramp merging scenario in this research consists of a single-lane entrance ramp merging onto a highway. The ego vehicle starts on the ramp approximately 150 m from the merging point, needing to merge with the vehicles on the highway and continue driving for at least 50 m after merging. This work establishes a traffic network with varying traffic densities in the simulation of urban mobility (SUMO) simulator [14]. In addition, a kinematic bicycle model is utilized to represent the motion dynamics of both the ego vehicle and surrounding vehicles, while the intelligent driver model (IDM) [15] is applied to simulate the driving behavior of the surrounding vehicles and their interactions with the ego vehicle, detailed as follows.

The kinematic bicycle model, as shown in Fig. 2, is used to simulate the motion of the ego vehicle, which can be described as follows [16], [17]:

$$\begin{cases} \dot{x} = v\cos(\psi + \beta) \\ \dot{y} = v\sin(\psi + \beta) \\ \dot{v} = a \\ \dot{\psi} = \dfrac{v}{l_r}\sin\beta \end{cases} \tag{1}$$

where x and y represent the longitudinal and lateral positions of the vehicle, respectively. v is the vehicle's speed, and a is the acceleration. l_r is the distance from the center of mass to the rear axle. ψ is the yaw angle, and β is the center of mass slip angle, which is related to the front wheel steering angle δ_f by the following equation:

$$\beta = \arctan\left(\frac{l_r}{l_f + l_r}\tan\delta_f\right) \tag{2}$$

where lf represents the distance from the center of mass to the front axle.

The motion of surrounding vehicles is simulated using IDM and the kinematic bicycle model. Specifically, the IDM simulates the longitudinal behavior of surrounding vehicles, such as their acceleration and following dynamics. The output of IDM, which is acceleration command a_{IDM}, is a continuous function of the controlled vehicle's speed v, the distance d between the controlled vehicle and the preceding vehicle, and the relative speed Δv between the controlled vehicle and the preceding vehicle, and it can be calculated using [18]

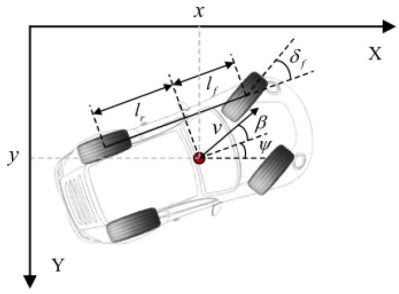

Fig. 2. Kinematic bicycle model of the vehicle.

$$a_{\text{IDM}} = a_{\max}\left[1 - \left(\frac{v}{v_0}\right)^{\delta} - \left(\frac{d^*(v,\Delta v)}{d}\right)^2\right] \tag{3}$$

where δ is the acceleration coefficient, and $d^*(v,\Delta v)$ is the desired distance between the controlled vehicle and the preceding vehicle, defined as follows:

$$d^*(v,\Delta v) = d_0 + Tv + \frac{v\Delta v}{2\sqrt{a_{\max}b}}. \tag{4}$$

The parameters of the IDM model include the desired speed v_0, the acceleration exponent δ, the minimum distance d_0, the safe time headway T, the maximum acceleration $a_{\max}$, and the desired deceleration b.

B. Problem Formulation

The motion planning task in this study aims to generate a safe, smooth, and efficient trajectory in on-ramp merging scenarios with dense traffic flow and frequent vehicle interactions. In this article, a trajectory is defined as τ, where τ represents the planned vehicle's trajectory over the time interval $[0, t_{\text{final}}]$. Under a specific cost function, the motion planning problem can be defined as follows:

$$\begin{aligned} &\tau^* = \arg\min_{\tau\in\Sigma} c(\tau) \\ &\tau(0) = x_{\text{init}} \\ &\tau(t_{\text{final}}) = x_{\text{final}} \\ &\tau(t) \in \mathcal{X}_{\text{free}} \forall t \in [0, t_{\text{final}}]. \end{aligned} \tag{5}$$

Σ represents the set of all feasible trajectories, τ^* denotes the optimal trajectory, and $\mathcal{X}_{\text{free}}$ indicates the safe space that avoids collisions with obstacles. The cost function $c(\tau)$ is designed to penalize trajectories that violate safety constraints, exhibit excessive jerk or acceleration, or fail to meet efficiency goals, ensuring that the planned trajectory is both practical and optimal.

III. Approach Overview

A. Dataset Collection

Connected vehicles possess the capability for continuous transmission driving data to the cloud, creating substantial potential value from the large datasets they generate. In this study, we aim to extract motion planning trajectories with the maximum utility from this type of available offline data. However, to guarantee the reproducibility of the experimental results,

Authorized licensed use limited to: MULTIMEDIA UNIVERSITY. Downloaded on December 08,2025 at 09:01:57 UTC from IEEE Xplore. Restrictions apply.

the offline dataset in this study was generated using SUMO, instead of relying on data from a real cloud. To reflect varying levels of driving behaviors in the real world, three strategies representing high-, medium-, and low-quality strategies were implemented in SUMO. The high-quality strategy emulates experienced drivers, producing smooth and collision-free trajectories; the medium-quality strategy reflects novice drivers, which may result in less optimal decisions; and the low-quality strategy intentionally introduces unsafe maneuvers, generating datasets with collisions. In addition, to further ensure that the collected data reflects the diverse and dynamic conditions that autonomous vehicles may encounter in real-world scenarios, we used random seeds in SUMO to simulate variations in traffic density, vehicle behavior, and environmental factors.

B. Data Processing

Once the data are generated from SUMO, the collected datasets are processed to produce a trajectory τ, which consists of a sequence of state s_T, action a_T, and reward r_T over a clipped period of T steps.

State: The vehicle state information s_t includes both the ego vehicle's state information s_e^t and the surrounding vehicles' state information s_i^t. The ego vehicle's state information s_e^t consists of four components: longitudinal position x_e^t, lateral position y_e^t, velocity v_e^t, and acceleration a_e^t. The state information for the surrounding vehicles s_i^t is composed of four components: longitudinal position x_i^t, velocity v_i^t, acceleration a_i^t, and whether the surrounding vehicle is present P_i^t. As shown in the on-ramp merging scenario in Fig. 1, the surrounding vehicle is traveling on a straight road, with its lateral position y_i^t being a fixed value. Therefore, when considering the state information of the surrounding vehicle, its lateral position is not considered. Overall, the state information can be represented as

$$\begin{cases} s_t = \left(s_e^t,\ s_i^t\right) \\ s_e^t = \left(x_e^t, y_e^t, v_e^t, a_e^t\right) \\ s_i^t = \left(x_i^t,\ v_i^t,\ a_i^t,\ P_i^t\right) \\ -L_1 < x_i^t < L_2,\ i \le 4. \end{cases} \tag{6}$$

In (6), L_1 and L_2 represent the detection distances in front of and behind the ego vehicle, respectively. The constraint condition $i \le 4$ indicates that only the information of the four nearest vehicles that satisfy $-L_1 < x_i^t < L_2$ is included in the surrounding vehicle state information s_i^t. If the number of surrounding vehicles that satisfy the constraint $-L_1 < x_i^t < L_2$ is n, and n is less than 4, then the remaining surrounding vehicles' state information s_i^t for $n+1 \le i \le 4$ will be filled with 0.

Action: The control action a_T represents the jerk value of the ego vehicle, which is the derivative of acceleration.

Reward: r_T is the reward given to state and action. The specific reward function is as follows:

$$r(s_t, a_t) = \begin{Bmatrix} r_{\text{crashed}} = -10 & \text{crashed} \\ r_{\text{succeed}} = 10 & \text{succeed} \\ r_{\text{racing}} & \end{Bmatrix} \tag{7}$$

where r_{crashed} is the reward when a collision occurs, r_{succeed} is the reward for successfully merging onto the ramp, and r_{racing} refers to the reward for comfort, efficiency, and safety as described in the following equations:

$$r_{\text{racing}} = \omega_1 - \begin{bmatrix} \omega_2 (v_t - v_{\text{desired}})^2 \\ +\omega_3 a^2 \\ +\omega_4 (\text{jerk})^2 \end{bmatrix} + \omega_5 r_d \tag{8}$$

$$r_d = \begin{cases} -\dfrac{2}{\max(d_{\min}, 1)} & d_{\min} < d_{\text{safety}} \\ 0 & d_{\min} \ge d_{\text{safety}} \end{cases} \tag{9}$$

where ω_1 is the weight that penalizes the time the ego vehicle spent stationary, ω_2 is the weight that penalizes the ego vehicle deviate from the desired speed, ω_3 and ω_4 represent comfort weights, and ω_5 imposes a penalty for approaching dynamic obstacles within a safe distance.

C. Transformer-Based Offline RL for Motion Planning

Given the offline data we collected, the motion planning problem can be considered a specific instance of offline RL. The goal in offline RL is to learn an optimal policy that can generalize well when deployed in the real world, despite the constraint of not being able to gather new data or interact with the environment during training. This makes offline RL useful in scenarios where real-time exploration is expensive, dangerous, or impractical, such as the on-ramp merging task in this research. Classical offline RL requires specialized techniques such as conservatism in policy learning (e.g., conservative Q-learning, CQL [19]) or incorporating uncertainty estimates [20] to avoid relying on out-of-distribution predictions. With a proper vehicle model, the policy learned from this type of offline RL can be used as a motion planning algorithm [1]. However, the compounding errors in this single-step model may lead to physically implausible planning. Inspired by Janner et al. [13], offline RL can also be tackled with the tool of sequence modeling, using a transformer architecture to maximize the expected cumulative rewards over trajectories and repurposing beam search as a motion planning algorithm.

In detail, our model employs a transformer decoder similar to the GPT architecture in [21]. Given that both our model and search strategy closely resemble those commonly used in NLP, and due to limitations in paper length, our focus is less on the transformer architectural design and more on how to effectively represent trajectory data by the discrete-token required. The process begins by discretizing the continuous states and actions. Discretization refers to the process of converting continuous physical quantities (e.g., position, velocity, and acceleration) into discrete values. Specifically, each dimension is independently converted into a sequence of discrete tokens achieved through uniform discretization, where all tokens for a given dimension correspond to a fixed width of the original continuous

Authorized licensed use limited to: MULTIMEDIA UNIVERSITY. Downloaded on December 08,2025 at 09:01:57 UTC from IEEE Xplore. Restrictions apply.

space. For example, assuming a per-dimension vocabulary size of V, the tokens for state dimension i cover uniformly spaced intervals of width $(\max s^i - \min s^i)/V$. Unlike the NLP tasks in which the transformer leverages a learned joint distribution over sequences to optimize predictions, we replace the transition log-probability with the reward signals [as detailed in (7)] to maximize the trajectory's expected returns. Furthermore, considering this reward-maximizing procedure carries the risk of leading to myopic behavior, where the model focuses too narrowly on the rewards within the prediction horizon, we augment each transition in the collected trajectories with another quantity which is returns-to-go, defined as $R_t = \sum_{i=t}^{T} \gamma^{i-t} r_i$, alongside immediate rewards r_t. This approach allows the transformer to generate actions based not only on rewards within the prediction horizon but also on the desired future returns, promoting long-term decision-making. After discretizing rewards r_t and returns-to-go R_t using a similar uniform discretization procedure as states and actions, these tokens are then fed into the transformer, where the self-attention mechanism captures the complex relationships between them, enabling the model to learn and predict future states, actions, and the corresponding reward. To balance between the exhaustive search of all possible solutions and the greedy search that might get stuck in suboptimal choices, the beam search as a sequence generation algorithm is used for motion planning. Beam search, a heuristic strategy, retains a set of top-scoring candidate sequences at each step of the H-step online planning process, ultimately identifying the highest scoring complete sequence.

D. Efficient Transformer for Motion Planning

The transformer architecture discussed earlier considers each dimension of the state value s, action value a, reward r, and returns-to-go R as individual tokens with trajectory decoding performed autoregressively across these tokens. While this approach effectively captures complex sequence dependencies, its computational intensity makes direct implementation infeasible on automotive-grade controllers with limited processing capabilities. To enhance the efficiency of transformer-based motion planning, our proposed efficient transformer planner employs two main strategies: efficient tokenization and temporal abstraction. We now describe the intuition and detailed theory behind these techniques.

Efficient tokenization is inspired by how human drivers process information, where they often consolidate multiple factors from the environment into a single decision-making unit [22]. For instance, a driver might simultaneously consider road conditions, traffic signals, nearby vehicle positions, their intended actions (e.g., accelerating and changing lanes), and the expected outcomes of these actions (e.g., safety and efficiency) as part of a single decision-making process rather than treating each factor in isolation. Analogously, we treat $\boldsymbol{x}_t := (s_t, a_t, r_t, R_t)$ as a single token in both the transformer encoder and decoder. Furthermore, the technique of temporal abstraction is also modeled after the way human drivers manage complex tasks [23]. Consider an overtaking scenario as an example: a human driver might first recognize the need to change lanes, plan an acceleration phase, and finally merge back into the original lane. Rather than precisely controlling each small action at every moment (such as the exact amount of acceleration and steering), the driver consolidates these steps into broader, high-level actions that span several time intervals. Drawing from this, our model also learns to abstract high-level features across multiple time steps, condensing detailed actions into broader higher level actions. This is achieved by abstracting the trajectory sequence into a latent space through vector quantized variational autoencoder (VQ-VAE) [24]. The VQ-VAE consists of three main components: an encoder that maps inputs spanning multiple time steps into discrete latent actions, a decoder that reconstructs inputs from those latent actions, and a learned prior distribution over the latent actions. Specifically, as illustrated in Fig. 3 and Table I, each token $\boldsymbol{x}_t$ is processed through a causal transformer encoder, producing a feature vector for each time step. These vectors are then pooled and passed through a linear layer with a kernel size

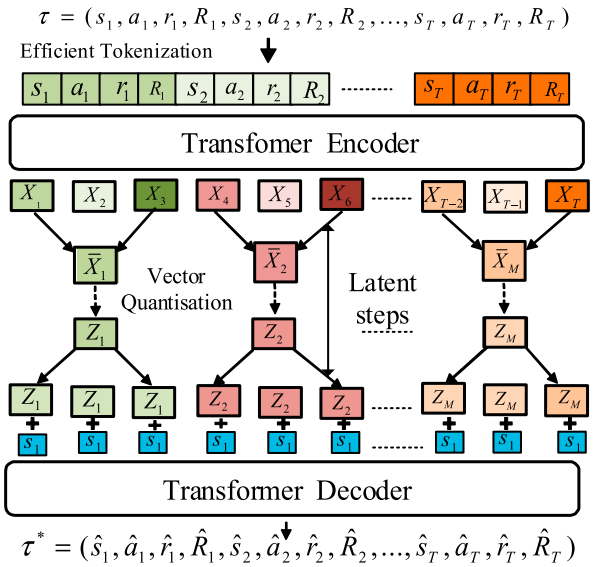

Fig. 3. Training process of efficient transformer planner.

TABLE I
PSEUDOCODE OF THE TRAINING PROCESS OF THE TOP ALGORITHM

Algorithm1: TOP Training.

Input: Training dataset D, Train epoch N, Batch size B, Encoder g, Decoder h, Latent steps L, Prior model $\boldsymbol{p}$, Sequence length $\boldsymbol{T}$
function : Training Encoder and Decoder (D, N)
  for epoch $= 0, 1, 2, \cdots, N$ **do**
    batch traj $\leftarrow$ sample batch traj (D, B):
    $\{x_1, x_2, \cdots, x_T\} \leftarrow$ efficient tokenization(batch traj)
    $\{X_1, X_2, \cdots, X_T\} \leftarrow g(\{x_1, x_2, \cdots, x_T\})$
    $\{Z_1, Z_2, \cdots Z_M\} \leftarrow$ temporal abstraction($\{X_1, X_2, \cdots, X_T\}$, L)
    reconstructed traj $\leftarrow h(\{Z_1, Z_2, \cdots Z_M\})$
    update $\boldsymbol{g}$, $\boldsymbol{h}$ by reconstruct loss
  end for
end function
function Training Prior model (D, N)
  for epoch $= 0, 1, 2, \cdots, N$ **do**
    $\boldsymbol{s}, \hat{Z}_i \leftarrow$ get state and target latent actions (D, B)
    $Z_i \leftarrow$ prior model($\boldsymbol{s}, Z_0, Z_1, \cdots Z_{i-1}$)
    update $\boldsymbol{p}$ by cross entropy loss (Z_i , $\hat{Z}_i$)
  end for
end function

Authorized licensed use limited to: MULTIMEDIA UNIVERSITY. Downloaded on December 08,2025 at 09:01:57 UTC from IEEE Xplore. Restrictions apply.

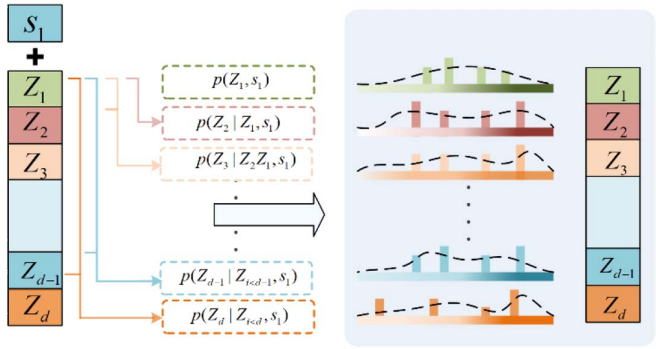

Fig. 4. Prior distribution of latent actions using a transformer.

and stride of L, reducing the sequence to $M = T/L$ vectors assuming the original feature vector consists of T elements. After that, these vectors undergo vector quantization, where each is compared to K embedding vectors $\boldsymbol{e}_k \in R^D$ in the embedding space $R^{K\times D}$, where $k \in 1, 2, 3, ..., K$. Finally, the closest match is selected, resulting in latent actions $(Z_1, Z_2, ..., Z_M)$. For the decoder, each latent action is repeated L times to match the length of the feature vector, while the state information is concatenated and projected to assist trajectory reconstruction. To capture the dependencies between latent actions, as illustrated in Fig. 4, we use a transformer model to learn the prior distribution of latent actions, autoregressively predicting their distribution based on the initial state s_1.

In contrast to the beam search in vanilla transformer, our scoring function, as defined in (10), operates in a latent action space, as illustrated in Fig. 5 and Table II, and introduces an additional penalty term $\alpha \ln\left(\min\left(p(z \mid s_1), \beta^M\right)\right)$ to handle out-of-distribution trajectories. The hyperparameter α is set to a value greater than the maximum discounted return to ensure that a reasonable trajectory is selected when the conditional probability of latent actions is below the threshold β^M. When the probability exceeds the threshold, the objective function encourages selecting plans with high-return

$$G(s_1, z) = \sum_t \gamma^t \hat{r}_t + \gamma^T \hat{R}_T + \alpha \ln\left(\min\left(p(z \mid s_1), \beta^M\right)\right). \quad (10)$$

Through efficient tokenization and temporal abstraction, we construct a compact latent space, which simplifies the search space and ultimately enables the identification of the optimal trajectory with significantly lower computational complexity. Specifically, in the vanilla transformer approach, each trajectory step is represented by $D = S + A + 2$ tokens, where S is the dimension of the state space and A is the dimension of the action space. Since the time complexity of the transformer for a sequence of length N is $O(N^2)$, modeling a trajectory with T steps requires TD tokens, leading to a time complexity of $O(D^2T^2)$. In contrast, the proposed efficient transformer planner models the trajectory with only T tokens, reducing the time complexity to $O(T^2)$.

E. Transformer Optimization Planner

As a data-driven method, the transformer-based algorithms inherently face challenges with the long-tail issue, making it difficult to accurately predict events that occur infrequently. To mitigate this issue, this research proposes a fine-tuning approach to constrain the ego vehicle's trajectories generated by the transformer within a safe zone while taking both physics-based and data-driven-based predictions of the surrounding vehicles into consideration. Specifically, we integrate the transformer model with the IDM to predict the longitudinal trajectories of the surrounding vehicles in on-ramp merging scenarios similar to the approach used by Geng et al. [25]. The transformer model captures complex driving behaviors of the surrounding vehicles, while the IDM imposes physical constraints to ensure the predicted trajectories align with vehicle kinematics. This integration of physical principles into the trajectory prediction process provides a more robust and accurate forecast even in unseen scenarios. Based on the improved prediction of surrounding vehicles, which serves as the boundary condition for the subsequent optimization process, the ego vehicle's trajectory is then refined using a QP method, as demonstrated in

$$\begin{aligned} \min_{s(t|i),\, i=j:k} J &= \sum_j^k \begin{bmatrix} \omega_1 (s_i - s_{rl}) \\ +\omega_2 (s_i'(t))^2 \\ +\omega_3 (s_i''(t))^2 \\ +\omega_4 (s_i'''(t))^2 \end{bmatrix} \\ \text{safe}_{\text{bound}_{\min}} &\le s_i \le \text{safe}_{\text{bound}_{\max}} \\ v_{\min} &\le s_i' \le v_{\max} \\ a_{\min} &\le s_i'' \le a_{\max} \\ j_{\min} &\le s_i''' \le j_{\max} \end{aligned} \quad (11)$$

where ω_1, ω_2, ω_3, and ω_4 are the weights for each term, s_{rl} is the ego vehicle trajectory provided by the transformer-based motion planning method, and $\text{safe}_{\text{bound}}$ refers to the safe driving space not occupied by the predicted surrounding traffic. By carefully tuning these weights, we achieve an effective tradeoff between safety, smoothness, and efficiency.

The optimization problem consists of two main components: the initial trajectory provided by the transformer and a smoothness cost, which is constrained by the derivatives of distance, velocity, and acceleration to ensure the trajectory complies with vehicle kinematics. In practice, the algorithm is implemented in Python, where CVXPY [26] is used to define and formulate the optimization problem, and OSQP [27] is utilized to efficiently solve the QP problem.

IV. Results and Discussions

We start our analysis by exploring the transformer-based approach that forms the basis of TOP's performance, followed by an evaluation of how integrated optimization methods further refine and improve the generated trajectories. Finally, we conclude with an ablation study to investigate the impact of key components such as efficient tokenization and temporal abstraction on motion planning performance.

Authorized licensed use limited to: MULTIMEDIA UNIVERSITY. Downloaded on December 08,2025 at 09:01:57 UTC from IEEE Xplore. Restrictions apply.

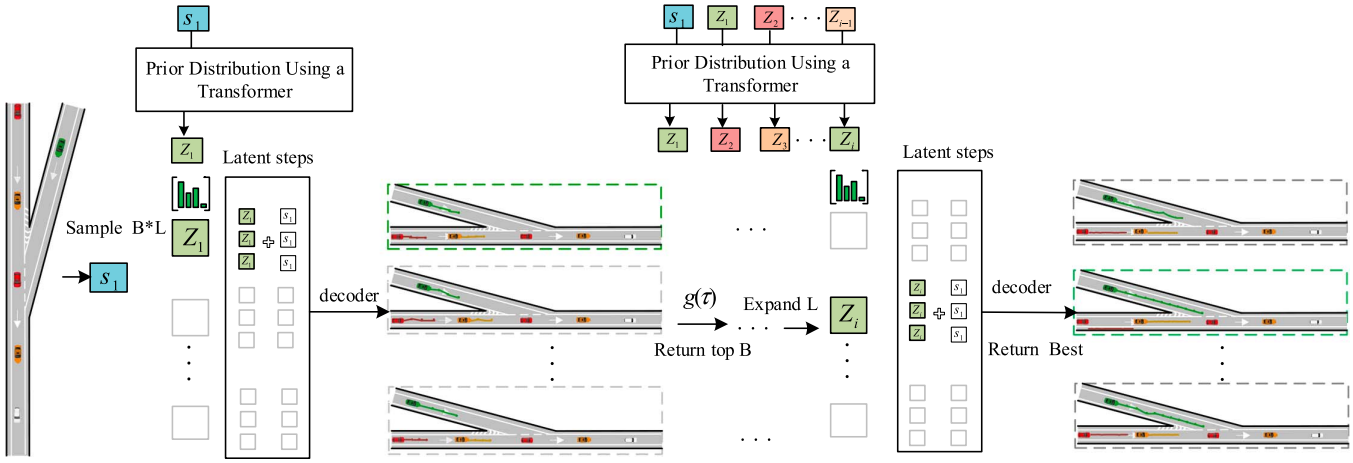

Fig. 5. Process of online planning in a temporally abstract latent action space using beam search in TOP.

A. Transformer-Based Motion Planner

In this part of the experiment, we compared the performance and time complexity of three motion planners: the vanilla transformer planner in IL setting, the vanilla transformer planner in offline RL setting, and the efficient transformer planner in offline RL setting. All planners were trained and executed on a PC equipped with an Intel i5-10500 processor and an NVIDIA GeForce GTX 1660 SUPER GPU. Hyperparameters of the algorithm, as shown in Table III, were used for all experiments.

The vanilla transformer planner, focused on the IL setting, aims to find the highest probability trajectory within the dataset. However, as indicated in Fig. 6, the highest probability trajectory results in the lowest accumulated rewards. This occurs because the planned trajectory tends to favor common but suboptimal actions found in the dataset rather than exploring less frequent actions that could potentially result in higher rewards. To ensure a fair comparison, all bar figures in this study were generated using the same ten random scenarios (with their corresponding random seeds in SUMO) as those used during dataset collection. These scenarios mimic variations in traffic density, vehicle behavior, and environmental factors, as outlined in Table IV. In each figure, the scatter points and the bar illustrate the individual and mean cumulative rewards for collision-free scenarios, with error bars representing the variation (mean $\pm$ standard deviation) between scenarios. The red line shows computational efficiency for vanilla and efficient transformer planners. In contrast, the vanilla transformer planner in the offline RL setting significantly improves accumulated rewards, but this comes at the expense of longer computation time, as incorporating reward signals increases the number of tokens the model must process. The efficient transformer planner outperforms both the vanilla transformer planner in the IL setting and the vanilla transformer planner in the offline RL setting in terms of accumulated rewards and computational efficiency. Notably, in terms of computational complexity, the vanilla transformer planner in the offline RL setting takes 1.6 seconds to execute, whereas the efficient transformer planner completes the same planning horizon in just 48.5 ms, achieving a 33-fold speed improvement. This satisfies the 100 ms real-time requirement for motion planning in emergency situations, as proposed by Fan et al. [3]. As outlined in Section III-D, the significant efficiency gains are primarily due to the transformer planner's integration of effective tokenization and temporal abstractions. This enables parallel trajectory generation in a more compact latent space, greatly reducing time complexity while preserving high-quality planning. Further details on the impact of tokenization and temporal abstraction on planning performance can be found in Section IV-C. In addition, we compared the test results in

TABLE II
PSEUDOCODE OF THE PLANNING PROCESS OF THE TOP ALGORITHM

Algorithm2 TOP Planning and Optimizing

Input: Current state $\boldsymbol{s}$, Planning horizon H, Beam width $\boldsymbol{B}$, Expansion factor $\boldsymbol{L}$, Prior model $\boldsymbol{p}$, Decoder h
function Online Planning (s, H, B, L, p, h)
  Sample initial context latents $Z_1 = \{z_1^{(n)} \mid z_1^{(n)} \sim p(z \mid \boldsymbol{s})\}, n \leq BL\}$
  for $t = 2, \cdots H$ **do**
    for $b = 1, 2, \cdots B$ **do**
      Expand the beam $C_t \leftarrow \{(z_{<t})^{(b)} \circ z_e \mid z_e \sim p((z_{<t} \mid z_{<t}^{(b)}, \boldsymbol{s}), e \leq L\}$
    end for
    Decode traj $\tau_t \leftarrow \{\tau = h(z_{<t+1}, \boldsymbol{s}), z_{<t+1} \in C_t\}$
    Get top B trajectories according to return
  end for
  best traj $\leftarrow$ trajectory with the highest return
  online optimizing trajectory (best traj)(Eq. 11)
  return optimized trajectory
end function

TABLE III
HYPERPARAMETERS OF THE TOP ALGORITHM

Hyperparameters	Value
learning rate	2e − 4
batch size	512
Number of attention heads	4
Number of Transformer layers	4
Hidden units	128
Embedding size for a latent code	512
Discount factor	0.99
β	0.05

Authorized licensed use limited to: MULTIMEDIA UNIVERSITY. Downloaded on December 08,2025 at 09:01:57 UTC from IEEE Xplore. Restrictions apply.

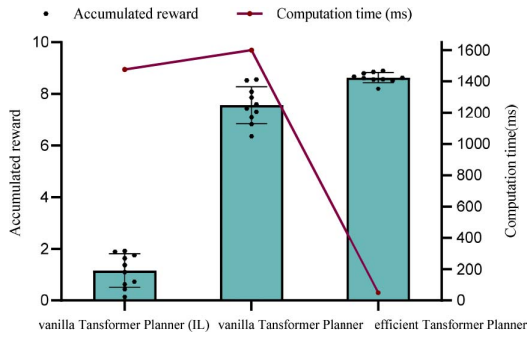

Fig. 6. Comparison of performance and computational efficiency for vanilla and efficient transformer planners.

TABLE IV
PARAMETERS DESCRIBING DIVERSE DRIVING BEHAVIOR OF SURROUNDING VEHICLES

Vehicle Type	Color	Accel	MinGap	MaxSpeed
aggressive	red	4.5	2.5	40
cautious	white	4.5	7.5	40
slow	yellow	3.0	2.5	20

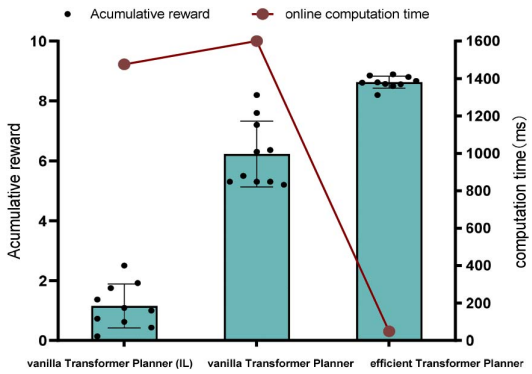

Fig. 7. Comparison of performance and computational efficiency for vanilla and efficient transformer planners in multilane merging scenarios.

multilane merging scenarios, as shown in Fig. 7. The traditional transformer method (e.g., vanilla transformer) shows reduced average reward, while efficient transformer planner demonstrates stronger robustness. This is due to the prior distribution-guided sampling mechanism (10), where the penalty term $\alpha \ln\left(\min\left(p(z \mid s_1), \beta^M\right)\right)$ encourages high-return trajectories while avoiding infeasible ones.

B. Transformer Optimization Planner

A transformer model trained on datasets that do not encompass all possible driving scenarios may struggle to generalize to unfamiliar situations, leading to poor decision-making in rare or complex environments. This increases the risk of unsafe behavior, such as misjudging vehicle trajectories or failing to recognize hazards, ultimately compromising the safety and reliability of the autonomous vehicle. For example, in our research, while the proposed efficient transformer planner performs well in most unseen testing scenarios, we did find out that in some cases, the ego vehicle's planned space-time (S-T) trajectory

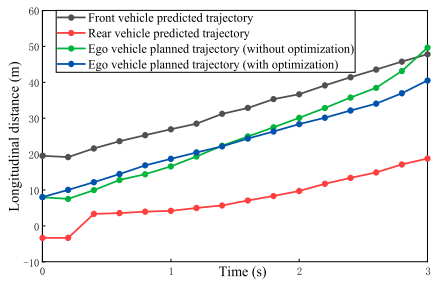

Fig. 8. Comparison of space-time (S-T) projections of the ego vehicle and surrounding traffic participants with the ST projection of the online optimized ego vehicle trajectory.

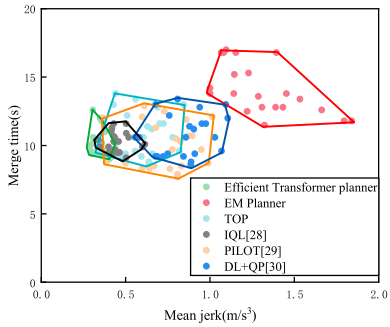

Fig. 9. Merging time and average jerk for different methods.

intersects with the predicted S-T paths of nearby participants, suggesting potential collisions, as shown in Fig. 8. To address these safety concerns, we implemented an online optimization process to refine the ego vehicle's trajectory, ensuring compliance with predefined safety requirements and kinematic constraints. After optimization, the ego vehicle's S-T trajectory (in blue) successfully avoids intersecting with the predicted paths of the nearby vehicles, preventing potential collisions. In addition to showcasing the safety improvements achieved through online refinement, it is also essential to compare our method with some well-established industrial algorithms to highlight its superior performance. The following will compare the proposed TOP and EM Planner using four evaluation metrics, including: 1) comfort, measured by jerk (the derivative of acceleration), where a higher jerk value indicates lower comfort; 2) traffic efficiency, defined as the average time taken for merging vehicles to complete the merging task; 3) safety, defined by the collision rate during the task, with a higher collision rate indicating lower safety; and 4) online computational efficiency, which is inversely proportional to the algorithm's execution time. Fig. 9 shows that TOP significantly outperforms EM Planner, highlighting its distinct advantages in driving comfort and traffic efficiency. This improved performance is primarily due to the TOP algorithm's ability to provide a better initial solution: under the guidance of the prior distribution, TOP selects more valuable driving trajectories during online planning, which ultimately enhances overall performance. A comparison was also conducted between the proposed method and two related approaches: RL-based planners [28] and hybrid methods [29], [30]. To ensure a comprehensive evaluation, we performed 25 experimental trials using identical hyperparameter settings, with

Authorized licensed use limited to: MULTIMEDIA UNIVERSITY. Downloaded on December 08,2025 at 09:01:57 UTC from IEEE Xplore. Restrictions apply.

Fig. 10. 1:10-Scale autonomous vehicle platform.

TABLE V
RESULTS OF 1:10 SCALE VEHICLE EXPERIMENTS

	TOP	EM Planner	Vanilla Transformer Planner
Merge Times ($\|s\|$)	7.49±1.41	11.65±1.85	9.45±1.72
Crash rate ($\|\%\|$)	0	0	7.3±0.65

the only variation being the random seed across all trials. Compared with the RL-based planner [28], for both the jerk and merge time metrics, the two-sample t-tests yielded $t = -4.8$, $p = 0.99$ for jerk and $t = -0.27$, $p = 0.6$ for merge time. The null hypothesis for both metrics was that our method would perform better than [28], and since the p-values exceeded 0.05, the results indicate that our method significantly outperforms [28] in both metrics. Compared with the hybrid methods [29] and [30], the two-sample t-test results show that for jerk ($t = -2.92$, $p = 0.99$; $t = -5.78$, $p = 1$), the proposed method is significantly lower than [29] and [30] (null hypothesis: the proposed method has lower jerk values than [29] and [30]). For merge time ($t = 1.49$, $p = 0.14$; $t = -0.01$, $p = 0.98$), the proposed method performs comparably (null hypothesis: no significant difference in merge time between the proposed method and [29] and [30]). These results indicate that the proposed method improves comfort while maintaining traffic efficiency. We have implemented the proposed method on a 1:10 scale intelligent vehicle platform, as illustrated in Fig. 10. The vehicle measures $425 \times 180 \times 200$ mm^3, with a full-load endurance of 2 h and a range of up to 8 km. The platform integrates the 4Pro intelligent driving suite to construct a multimodal perception-decision system. The results of 25 repeated experiments, summarized in Table V, show consistency with the simulation outcomes. To further validate the real-time performance of our algorithm, we tested it on the Apollo D-KIT platform, equipped with an NVIDIA GeForce RTX 2070S GPU. The results, as shown in Fig. 11, demonstrate that TOP achieves an average execution time of approximately 38.5 ms, which is more than three times faster than EM Planner, which took 128 ms, and 33 times faster than the transformer-based planner, which had an average execution time of 1270 ms. Additionally, safety was evaluated through 25 trials with randomized traffic flows. Both TOP and EM Planner achieved a 0% collision rate, outperforming the transformer planner, which had an average 5.3% collision rate

TABLE VI
COMPARISON OF TOP, EMPLANNER, AND TRANSFORMER-BASED PLANNER PERFORMANCE

	TOP	EM Planner	Vanilla Transformer Planner
Merge Times ($\|s\|$)	10.74±1.65	14.15±1.78	10.22±0.72
Jerk($\|m/s^3\|$)	0.60±0.13	1.31±0.24	0.55±0.047
Crash rate($\|\%\|$)	0	0	5.3±0.31

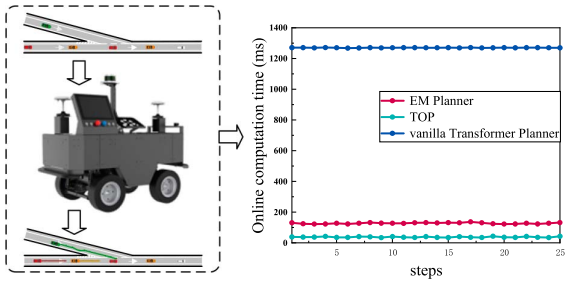

Fig. 11. Real-time performance comparison on Apollo D-KIT platform.

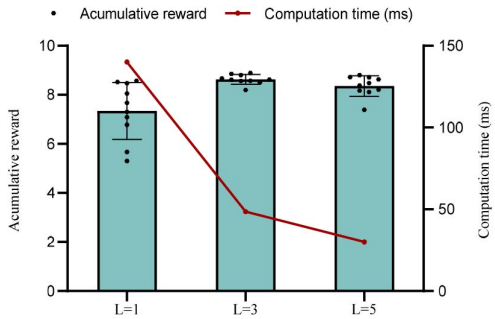

Fig. 12. Comparison of performance across different levels of temporal abstraction.

($\sigma = 0.31$), as shown in Table VI. A two-sample t-test comparing TOP & EM Planner with the transformer planner yielded $t = -85.5$, $p < 0.001$, rejecting the null hypothesis and confirming TOP's superior safety performance.

1) Ablation Study:

a) The impact of efficient tokenization and temporal abstraction on the TOP: Efficient tokenization and temporal abstraction are key features of the TOP algorithm. Efficient tokenization reduces the complexity of the input data by consolidating multiple features into a single token, while temporal abstraction leverages latent steps (L) to capture patterns across multiple time steps. However, the impact of these design choices on planning performance needs further investigation. As shown in Fig. 12, When L is set to 1, only the technique of efficient tokenization is applied, resulting in a faster runtime of 140 ms, compared to the 1.6 seconds required by the vanilla transformer planner (as shown in Fig. 6). While the runtime compared to the vanilla transformer planner is significant, with $L = 1$, each latent action decodes only a single time step, limiting the model's ability to generalize and capture higher level temporal patterns over multiple time steps, which can lead to reduced planning flexibility. In addition, this fine-grained fitting may also lead to

Authorized licensed use limited to: MULTIMEDIA UNIVERSITY. Downloaded on December 08,2025 at 09:01:57 UTC from IEEE Xplore. Restrictions apply.

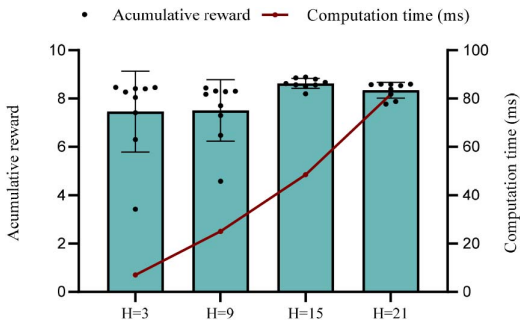

Fig. 13. Comparison of performance for different online planning steps.

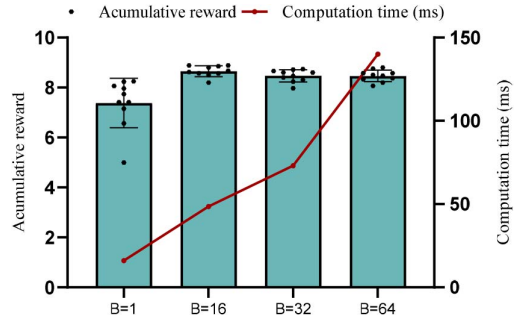

Fig. 14. Comparison of performance for different beam width.

overfitting by concentrating on individual transitions, which likely contributes to the performance decline. When L is increased to 5, the level of temporal abstraction is enhanced, but the performance experiences a slight decline. This occurs because, with $L = 5$, the trajectory decoded from a single latent code spans too many time steps. As a result, any initial prediction error propagates through the following steps, leading to an accumulation of errors over time. To balance the balance between performance and computational complexity, the present study selects $L = 3$ as the optimal setting.

b) The impact of online planning steps on the TOP: In this section, we examined how different online planning steps H influence the algorithm's performance. Specifically, four different values (H: 3, 9, 15, and 21) were chosen for the online planning steps.

As shown in Fig. 13, performance is lowest when $H = 3$, likely because fewer planning steps fail to sufficiently account for future dynamic changes and interactions, limiting decision accuracy. Performance peaks at $H = 15$, indicating that this step count effectively considers the impact of future states and actions, which enhances decision accuracy and stability. However, at $H = 21$, performance slightly declines. This is because, while additional planning steps can capture longer term dynamic information, too many steps may introduce unnecessary complexity or noise, reducing overall performance. It is also worth noting that as H increases, the online computation time also rises, as depicted by the red line in Fig. 11. To balance performance and computational efficiency, $H = 15$ is chosen as the optimal setting in this research.

c) The impact of beam search the TOP: Beam search allows for the expansion of the search space, enhancing the likelihood of discovering the optimal solution. When the beam width $B = 1$, the performance of TOP significantly decreases. This is due to the algorithm relying exclusively on autoregressive sampling from the prior policy to generate action recommendations. While this method generates the most probable actions for each state, the sequence of these actions may not be globally optimal. As shown in Fig. 14, increasing beam width significantly improves performance, peaking at $B = 16$. However, increasing the beam width to $B = 32$ and $B = 64$ introduces slight performance decline and higher computational costs. To balance performance and computational efficiency, $B = 16$ is chosen as the optimal setting for this research.

V. Conclusion

In this study, using the example of autonomous vehicles merging onto highways from on-ramps, we propose a novel trajectory planning algorithm named TOP. The method integrates the sequence modeling capabilities of transformers with offline RL to derive planning trajectories from precollected datasets. To meet real-time planning requirements, efficient tokenization and temporal abstraction are applied, reducing the complexity of motion planning by efficiently tokenizing multidimensional environmental information into a single decision-making unit and compressing input sequences into high-level latent actions. These high-level actions result in more humanlike planned trajectories, which are subsequently refined through optimization techniques to ensure adherence to predefined safety requirements and vehicle kinematic constraints. We validated TOP's performance through comprehensive experiments, comparing it with both optimization-based and vanilla transformer-based planners. The results show that TOP significantly outperforms benchmark methods in terms of trajectory smoothness, safety, and computational efficiency while remaining feasible for direct implementation on automotive-grade controllers with limited processing capabilities. Building on these results, future research could focus on optimizing the TOP algorithm's application across a broader range of scenarios and exploring its performance in real-world road testing. Additionally, incorporating the impact of multimodal prediction with TOP could further improve the algorithm's adaptability and effectiveness.

References

[1] S. Zhang, W. Zhuang, B. Li, K. Li, T. Xia, and B. Hu, "Integration of planning and deep reinforcement learning in speed and lane change decision-making for highway autonomous driving," *IEEE Trans. Transp. Electrific.*, vol. 11, no. 1, pp. 521–535, Feb. 2025.

[2] F. Bouchard, S. Sedwards, and K. Czarnecki, "A rule-based behaviour planner for autonomous driving," in *Proc. Int. Joint Conf. Rules Reasoning*, Cham: Springer Int. Publishing, 2022, pp. 263–279.

[3] H. Fan et al., "Baidu Apollo em motion planner," 2018, *arXiv:1807.08048*.

[4] S. Wang, Z. Li, B. Wang, and M. Li, "Collision avoidance motion planning for connected and automated vehicle platoon merging and splitting with a hybrid automaton architecture," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 2, pp. 1445–1464, Feb. 2024.

[5] V. Bhattacharyya and A. Vahidi, "Automated vehicle highway merging: Motion planning via adaptive interactive mixed-integer MPC," in *Proc. Am. Control Conf.*, Piscataway, NJ, USA: IEEE, 2023, pp. 1141–1146.

[6] R. S. Sutton, "Reinforcement learning: An introduction," in *A Bradford Book*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

Authorized licensed use limited to: MULTIMEDIA UNIVERSITY. Downloaded on December 08,2025 at 09:01:57 UTC from IEEE Xplore. Restrictions apply.

[7] L. Chen et al., "What data do we need for training an AV motion planner?" in *Proc. IEEE Int. Conf. Robot. Autom.*, Piscataway, NJ, USA: IEEE, 2021, pp. 1066–1072.
[8] A. Vaswani, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, p. 30.
[9] J. Mao et al., "GPT-Driver: Learning to drive with GPT," 2023, *arXiv:2310.01415*.
[10] K. Cobbe et al., "Training verifiers to solve math word problems," 2021, *arXiv:2110.14168*.
[11] J. Ngiam et al., "Scene Transformer: A unified architecture for predicting multiple agent trajectories," 2021, *arXiv:2106.08417*.
[12] Z. Huang, H. Liu, and C. Lv, "Gameformer: Game-theoretic modeling and learning of transformer-based interactive prediction and planning for autonomous driving," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 3903–3913.
[13] M. Janner, Q. Li, and S. Levine, "Offline reinforcement learning as one big sequence modeling problem," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 1273–1286.
[14] D. Krajzewicz, "Traffic simulation with SUMO–simulation of urban mobility," in *Fundam. Traffic Simul.*, J. Barceló, Ed., vol. 145, New York, NY, USA: Springer, 2010, pp. 269–293.
[15] A. Kesting, M. Treiber, and D. Helbing, "Enhanced intelligent driver model to assess the impact of driving strategies on traffic capacity," *Philos. Trans. Roy. Soc. A: Math., Phys. Eng. Sci.*, vol. 368, no. 1928, pp. 4585–4605, 2010.
[16] P. Polack, F. Altché, B. d'Andréa-Novel, and A. de La Fortelle, "The kinematic bicycle model: A consistent model planning feasible trajectories autonomous vehicles?" in *Proc. IEEE Intell. Vehicles Symp. IEEE*, 2017, pp. 812–818.
[17] X. Tang, B. Huang, T. Liu, and X. Lin, "Highway decision-making and motion planning for autonomous driving via soft actor-critic," *IEEE Trans. Veh. Technol.*, vol. 71, no. 5, pp. 4706–4717, May 2022.
[18] M. Treiber, A. Hennecke, and D. Helbing, "Congested traffic states in empirical observations and microscopic simulations," *Phys. Rev. E*, vol. 62, no. 2, pp. 1805–1824, Aug. 2000.
[19] A. Kumar et al., "Conservative Q-learning for offline reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1179–1191.
[20] B. Hu, Y. Xiao, S. Zhang, and B. Liu, "A data-driven solution for energy management strategy of hybrid electric vehicles based on uncertainty-aware model-based offline reinforcement learning," *IEEE Trans. Ind. Informat.*, vol. 19, no. 6, pp. 7709–7719, Jun. 2023.
[21] A. Radford, "Improving language understanding by generative pre-training," 2018.
[22] M. R. Endsley, "Toward a theory of situation awareness in dynamic systems," *Hum. Factors*, vol. 37, no. 1, pp. 32–64, 1995.
[23] R. S. Sutton, D. Precup, and S. Singh, "Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning," *Artif. Intell.*, vol. 112, nos. 1–2, pp. 181–211, 1999.
[24] A. Van Den Oord and O. Vinyals, "Neural discrete representation learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, p. 30.
[25] M. Geng et al., "A physics-informed Transformer model for vehicle trajectory prediction on highways," *Transp. Res. Part C: Emerg. Technol.*, vol. 154, 2023, Art. no. 104272.
[26] S. Diamond and S. Boyd, "CVXPY: A Python-embedded modeling language for convex optimization," *J. Mach. Learn. Res.*, vol. 17, no. 83, pp. 1–5, 2016.
[27] B. Stellato et al., "OSQP: An operator splitting solver for quadratic programs," *Math. Program. Comput.*, vol. 12, no. 4, pp. 637–672, 2020.
[28] I. Kostrikov, A. Nair, and S. Levine, "Offline reinforcement learning with implicit Q-learning," 2021, *arXiv:2110.06169*.
[29] H. Pulver, F. Eiras, L. Carozza, M. Hawasly, S. V. Albrecht, and S. Ramamoorthy, "PILOT: Efficient planning by imitation learning and optimisation for safe autonomous driving," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Piscataway, NJ, USA: IEEE, 2021, pp. 1442–1449.
[30] J. Ichnowski et al., "Deep learning can accelerate grasp-optimized motion planning," *Sci. Robot.*, vol. 5, no. 48, 2020, Art. no. eabd7710.

Ping Lu was born in Shandong, China, in 2000. He received the Bachelor of Engineering degree in automobile service engineering from Guiyang University, Guiyang, China, in 2022. He is currently working toward the postgraduate degree with the Key Laboratory of Advanced Manufacturing Technology for Automobile Parts, Ministry of Education, Chongqing University of Technology, Chongqing, China.

His research interests include autonomous vehicle, trustworthy reinforcement learning, and decision-making under uncertainty.

Hui Xu was born in Sichuan, China, in 1998. He received the Bachelor of Engineering degree in vehicle engineering from Chengdu University, Chengdu, China, in 2022. He is currently working toward the postgraduate degree with the Key Laboratory of Advanced Manufacturing Technology for Automobile Parts, Ministry of Education, Chongqing University of Technology, Chongqing, China.

His research interests include autonomous vehicle and deep reinforcement learning.

Bo Hu was born in Hefei, Anhui, China, in 1989. He received the B.S. degree from Chongqing University of Technology (CQUT), Chongqing, China, in 2011, and the M.S. and Ph.D. degrees from the University of Bath, Bath, U.K., in 2012 and 2016, respectively, all in automotive engineering.

Currently, he is an Associate Professor with the Key Laboratory of Advanced Manufacturing Technology for Automobile Parts, Ministry of Education, CQUT. His research interests include machine learning-based control of intelligent and connected vehicles and modeling and control of advanced boosted engine systems.

Authorized licensed use limited to: MULTIMEDIA UNIVERSITY. Downloaded on December 08,2025 at 09:01:57 UTC from IEEE Xplore. Restrictions apply.